

2026년 개인주제탐구발표대회 보고서

파이썬 기초 문법으로 구현한
Q-Learning 기반 미로 탐색 AI

2026. 9. 4.

과 정 명	SW고급
소속 학교	유성중학교
성 명	오아인

대전교육정보원정보영재교육원

목차

I. 탐구의 필요성 및 목적

1. 탐구의 필요성
2. 탐구 목적

II. 이론적 배경

1. 용어 정의

III. 탐구 방법 및 과정

1. 탐구 환경
2. 주요 설계
3. 탐구 절차

IV. 연구 결과

1. 각 모델별 이동 수 그래프
2. 탐구 과정에서 발견한 문제
3. 문제 해결
4. 탐구 결과
5. 결과 분석

V. 결론 및 제언

1. 탐구 결론
2. 제언

※참고문헌

I. 탐구의 필요성 및 목적

1. 탐구의 필요성

완성된 AI 모듈을 이용하여 AI의 판단 과정을 이해하기 어려움. 특히 AI의 판단 기준인 보상과 판별식을 공부하기 힘들. 따라서 파이썬 기초 문법만을 활용하여 AI를 작성해보며 알고리즘을 이해할 필요가 있음

2. 탐구의 목적

1) 파이썬 기초문법으로 AI 구현

파이썬 기초 문법만으로 외부 라이브러리를 쓰지 않고 AI를 구현한다.

2) 보상에 따른 AI의 판단 결과 변화 관찰

보상의 차이에 따라 AI가 내리는 판단에 어떤 변화가 생기는지 관찰

II. 이론적 배경

3.이론적 배경

1)강화 학습: AI를 학습시키는 방법 중 하나로 AI가 자신의 환경을 인식하여 직접 학습하는 방법. 지도학습과는 다르게 정답을 알려 주지 않아 AI가 스스로 시행착오를 통해 학습함.

2) Q_learning: AI가 특정 상태(a)에서 행동(b)을 하였을 때 결과를 $Q(a,b)$ 로 저장하여 행동에 따른 결과를 학습하는 강화학습 방법

3) BFS 알고리즘: 그래프나 미로에서 시작점부터 인접한 칸으로 한 칸씩 탐색해 최단 경로를 찾는 대표적인 방법

III. 탐구 방법 및 과정

4.탐구 환경

1) 개발 언어: Python

2) AI 학습 모듈: 미사용

3) 그래픽: Pygame

4) 주요 자료

ㅏ.ai_scores: 각각의 칸에서 AI의 현재 위치를 Key로 가지고

이동 점수를 Value로 가진 딕셔너리들이 들어있는
이중 딕셔너리

↳.maze: 벽과 길, 출구를 0,1,2로 저장한 미로 리스트.

5. 탐구 절차

- 1) 기본 Q_table 구조 설계 및 pygame을 활용한 그래픽 만들기
- 2) AI의 제자리걸음 문제 해결
- 3) BFS 알고리즘을 이용하여 AI가 학습한 결과가 맞는지 확인
- 4) 실험 데이터 정리 및 보고서 작성

6. 주요 설계

first_ai 모델:

미로: 5x5 크기의 미로에 벽과 출구 만들고 AI를 시작 위치로 이동.

행동: 15%의 확률로 무작위 이동 및 85%의 확률로 학습 결과 대로 이동

보상: 화면 밖을 벗어나거나 벽에 충돌할 경우 -5.0점, 출구에 도착한 경우 50.0점을 줌.

학습: 일반 길을 이동할 경우: 이동 보상을 추가하지 않아 다음 위치의 최대 Q점수와 학습률 0.1, 할인율 0.9 만을 적용하여 Q점수를 업데이트.

시각화: Pygame으로 미로와 AI를 그리고 도착 횟수와 현재 걸음 수를 표시.

first_ai 모델의 Q-score Update 식

$\text{max_future_score} = \max(\text{ai_scores}[\text{next_state}].\text{values}())$

$\text{old_value} = \text{ai_scores}[\text{state}][\text{action}]$

$\text{ai_scores}[\text{state}][\text{action}] = \text{old_value} + \text{learning_rate} * (\text{discount_factor} * \text{max_future_score} - \text{old_value})$

second_ai 모델:

미로: 1단계 모델과 동일

보상: AI가 이동하는 경우 이동 보상으로 -0.1을 추가함. 이로 인해

제자리걸음이나 긴 우회 경로로 이동할수록 점수가 감점되어 AI가 하지

않음.

```
# second_ai 모델의 Q-score Update 식
max_future_score = max(ai_scores[next_state].values())
old_value = ai_scores[state][action]
reward = -0.1
ai_scores[state][action] = old_value + learning_rate *
(reward + discount_factor * max_future_score - old_value)
```

final_ai 모델:

미로: 무작위로 전체 칸 수의 25%만큼 벽을 세움.

최소 이동 수 계산: 시작부터 출구까지의 최소 이동 수를 계산. 만약 출구에 도달할 수 없는 미로가 생성된 경우 미로를 다시 생성.

최소 이동 수 계산

```
while len(search_list) > 0:
```

```
    row, col, steps = search_list.pop(0)
```

```
    if row == maze_size - 1 and col == maze_size - 1:
```

```
        return maze, steps
```

무작위 이동: 승리 횟수가 증가할수록 무작위 이동 확률을 줄임. 특정 횟수(미로 크기 2배) 이상 승리 시 무작위 이동 확률을 0으로 바꿔 학습된 데이터로만 이동하도록 설계.

레벨: 최근 3번의 걸음 수 평균과 최소 이동 수가 일치하면 팝업을 띄워 다음 레벨로 넘어갈지 결정. 또한 키보드 조작을 통해 미로 재생성과 크기 조절을 할 수 있음.

보상: second_ai와 동일

```
# final_ai Q-score Update 식
```

```
max_future_score = max(ai_scores[next_state].values())
```

```
old_value = ai_scores[state][action]
```

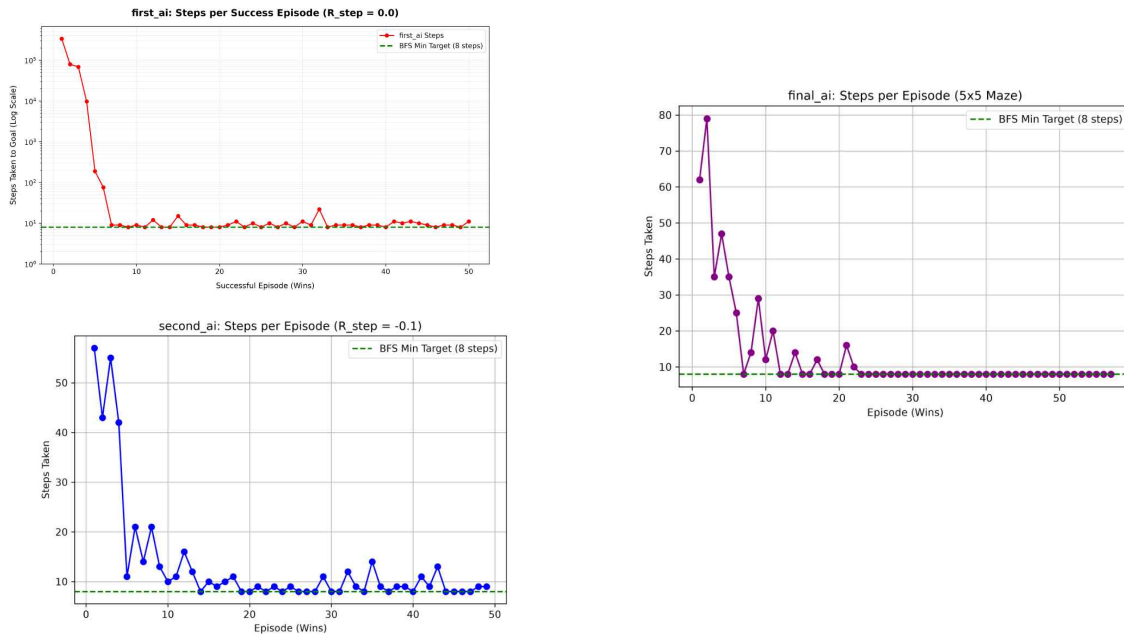
```
reward = -0.1
```

```
ai_scores[state][action] = old_value + learning_rate *
```

```
(reward + discount_factor * max_future_score - old_value)
```

IV. 연구 결과

7.각 모델별 이동 수 그래프(도착 50번, 5x5 미로 기준)



8.탐구 과정에서 발견한 문제

- 1) first_ai에서 AI가 이동 시 두 칸 사이를 무한히 왕복함.
- 2) second_ai에서 AI가 학습을 완료한 후에도 탐험을 계속하여 불필요한 이동을 함.
- 3) final_ai에서 무작위로 벽을 세우다 보면 벽이 시작점이나 출구를 완전히 둘러싸서
출구로 가는 길이 아예 막히는 경우가 있음.

9.문제 해결

- 1) first_ai 제자리걸음 해결방법: 이동 보상을 추가해 AI가 최대한 빠르게 목적지로 도달하도록 유도.

해결 코드

reward = -0.1

ai_scores[state][action] = old_value + learning_rate *

(reward + discount_factor * max_future_score - old_value)

- 2) second_ai 문제 해결 방법: 도착 횟수가 늘어날수록 무작위 이동 확률을 줄여

나중에는 학습된 데이터만 이용해서 이동하게 함.

해결 코드

```
random_rate = 0.2 - (win_count * 0.02)
```

```
if random_rate < 0.05:
```

```
    random_rate = 0.05
```

3) final_ai 미로 생성 문제 해결 방법: 미로 생성 직후 BFS 알고리즘을 이용해 출구에 도착 가능한 길이 있을 때만 미로를 보여주고 불가능한 경우 미로를 다시 생성

해결 코드

```
while len(search_list) > 0:
```

```
    row, col, steps = search_list.pop(0)
```

```
    if row == maze_size - 1 and col == maze_size - 1:
```

```
        return maze, steps
```

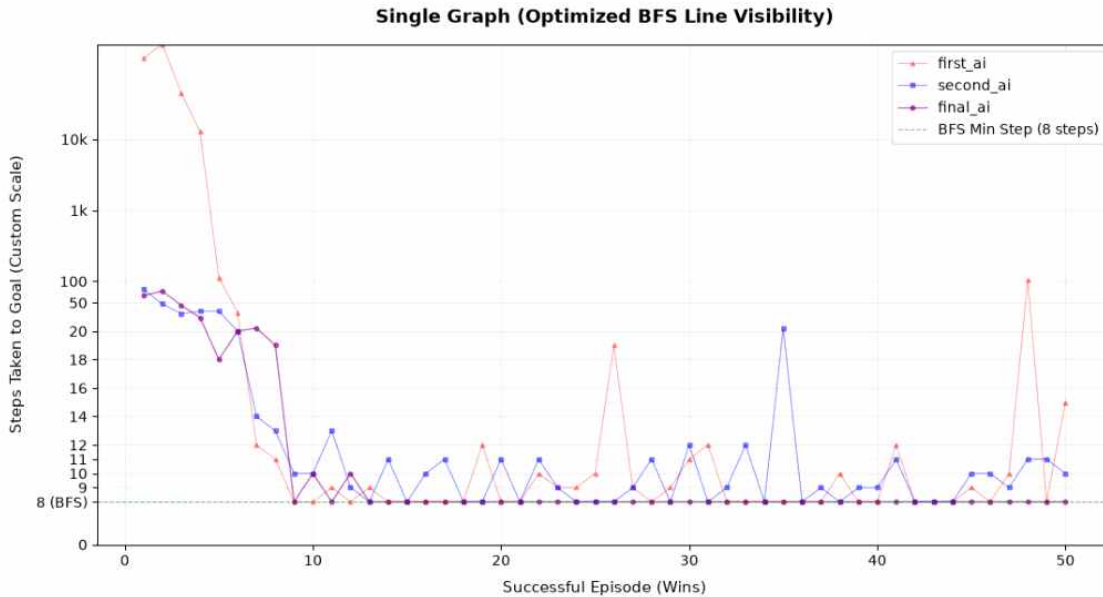
10.탐구 결과

1) python 기초 문법만으로 AI 개발에 성공함.

2) 이동 보상이 없을 경우 AI가 제자리걸음을 하며 제대로 학습되지 않으며 이동 보상을 추가 할 경우 AI가 최단 경로를 학습하고 이동함.

3) final_ai에서 평균 이동 수로 AI 학습 과정을 잘 알 수 있음. 또한 AI의 이동 수가 최소 이동 수와 일치함을 확인함.

11.결과 분석



first,second,final_ai의 이동 걸음 수를 비교한 그래프
위 그래프를 통해 이동 보상이 없을 경우 학습이 매우 비효율적임과 무작위
이동 확률을 조정하지 않을 경우 학습 완료 후에도 다른 길로 돌아갈 수 있음을
알 수 있음.

V. 결론 및 제언

12. 탐구 결론

파이썬 기초 문법만으로 AI 구현 성공

외부 라이브러리를 사용하지 않고 파이썬 기초 문법만으로 Q-Table 구조의 AI를
구현

외부 모듈 대신 미로 각 칸의 위치를 Key로 갖고 이동 방향에 따른 점수를
Value로 갖는 이중 딕셔너리를 설계함. 이를 통해 AI가 미로 내에서 자신의
행동을 효율적으로 판단할 수 있었음.

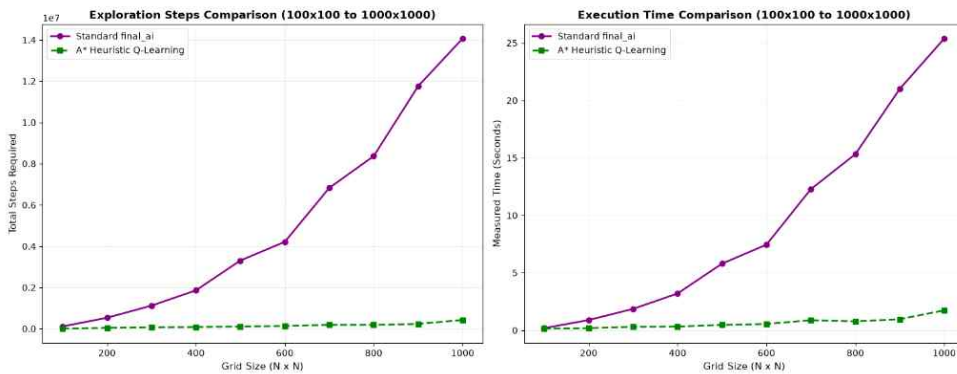
first_ai: 이동 보상이 없을 때 AI가 제자리걸음을 하며 학습 효율이 매우 낮음을
확인.

second_ai: 이동 보상을 추가하여 AI가 스스로 최단 경로로 이동하도록 설계.

final_ai: 승리 횟수에 따라 무작위 이동 확률을 감소시키는 알고리즘 추가 및
BFS 알고리즘을 이용하여 AI가 최적의 경로로 이동함을 확인.

13.제언

1) 크기가 큰 미로에서도 효율적인 A 휴리스틱 기반 Q-러닝 알고리즘도 탐구하고 싶음



final_ai와 A 휴리스틱 방식 알고리즘의 걸음 수 및 시간 비교 그래프 위 그래프처럼 단순 Q-러닝 방식으로는 미로 크기가 커 질수록 걸음 수와 시간이 기하급수적으로 증가함. 반면 A 휴리스틱 방식 알고리즘의 경우 비슷하게 유지됨.

이처럼 큰 미로에서도 효율적으로 학습하는 A 휴리스틱 방식 알고리즘도 탐구하고 싶음.

2) 움직이는 장애물 환경에서도 개발하고 싶음

이번 탐구에서는 벽과 길이 고정되어 있는 미로에서 강화학습을 진행함. 그러나 실제 자율주행이나 로봇 제어에서는 보행자와 다른 차량 등 위치가 변하는 장애물이 존재함. 앞으로는 움직이는 장애물 환경에서의 강화학습 알고리즘을 탐구해 보고 싶음

*참고 자료

<https://velog.io/@hamsangjin/PCCP-BFS-%EB%AF%B8%EB%A1%9C%EC%9D%98-%EC%B5%9C%EB%8B%A8%EA%B1%B0%EB%A6%AC-%ED%86%B5%EB%A1%9C>

-[PCCP] BFS - 미로의 최단거리 통로 ! 파이썬

<https://velog.io/@myway00/063-%EB%AF%B8%EB%A1%9C%EC%9D%98-%EC%B5%9C%EB%8B%A8%EA%B1%B0%EB%A6%AC-%ED%86%B5%EB%A1%9CBFS-%ED%99%9C%EC%9A%A9> -파이썬 알고리즘 063 ! 미로의 최단거리 통로(BFS 활용)@**

https://velog.io/@s_s/%EA%B0%95%ED%99%94%ED%95%99%EC%8A%B5-%EA%B8%B0%EC%B4%88-4-%ED%81%90%EB%9F%AC%EB%8B%9D -강화학습 기초 4 - 큐 러닝

<https://bluediary8.tistory.com/18> -예제로 쉽게 알아보는 강화학습 기초 (Q-learning, Reinforcement Learning)